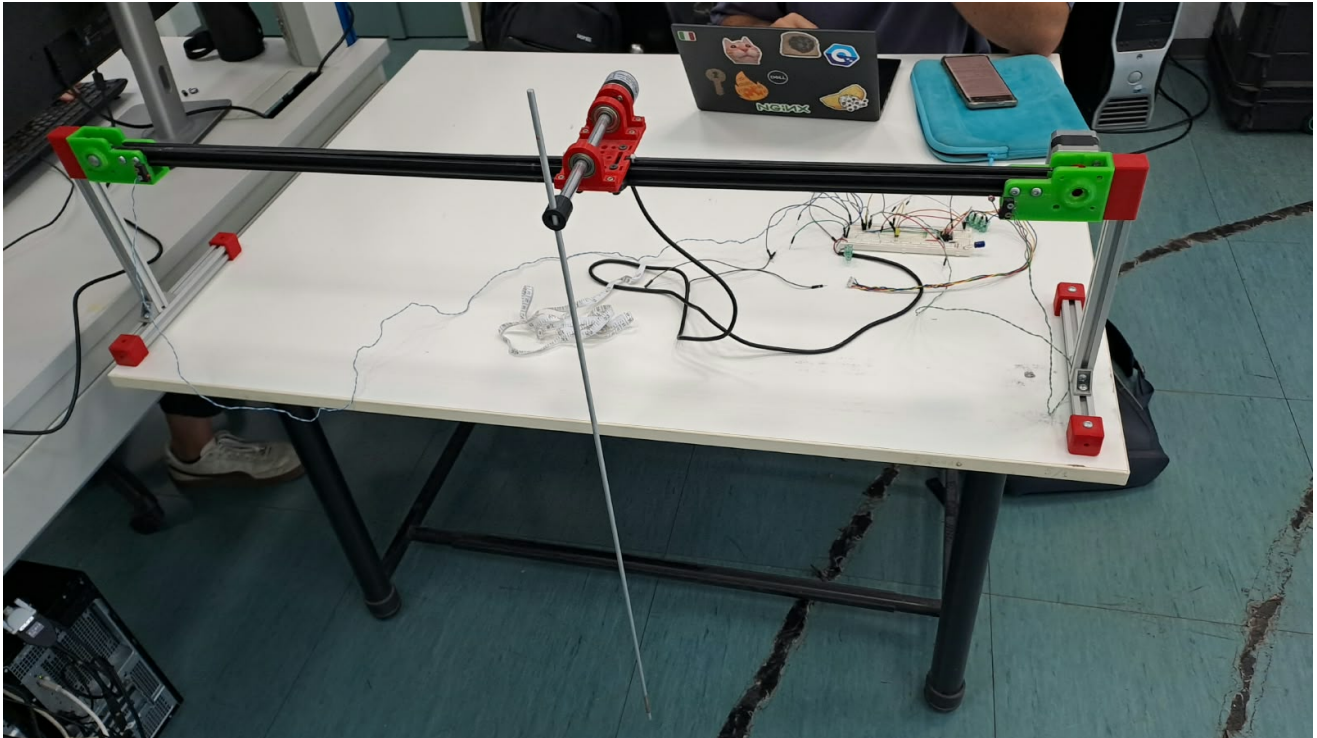


Your STM32 inverted pendulum

UPDATED PROJECT GUIDE / 8 OCTOBER 2026

Swing-up, upright balance, end stops and adjustable-pivot commissioning



Supplied plant photo; it is a reference image, not evidence of the present wiring.

Current verdict

Swing-up and LQR are implemented in the source. The current startup cannot reach control because homing ignores its switches. Several further sensing and control issues need correction before an automatic plant run.

Prepared for Lorenzo and the project team. This revision replaces the September guide as the description of the current source. It explains what exists, what has improved, what remains to fix, and what must be measured on the actual mechanism.

Validation: fresh ARM compile and link succeeded; focused host checks reproduced the key software defects; controller gains were independently recalculated. No firmware was flashed and no hardware operation or successful swing-up is claimed.

Firmware snapshot: Embedded-Course-Dma/pendulum/test. Source files are unchanged by this review. Original guide archived separately.

1. Read this first

The answer to “does it now manage swing-up?”

Yes at the source-code level: swingup.c computes an energy-based acceleration command, and the TIM3 callback switches between swing-up and LQR. No at the validated-system level: startup gets stuck at failed homing, the DMA sensing route is invalid, and several control prerequisites are unfinished.

Priority	Action required before automatic operation
1 - motion	Assign and configure both end stops; restore their reads; add runtime limit supervision and a fault stop. Homing currently commands motion with no switch feedback.
2 - sensing	Replace or redesign DMA sampling; calibrate encoder zero/sign/counts; initialize the first velocity sample and filter the estimate.
3 - swing-up	Call swingup_init; provide measured inertia and COM distance; verify the energy-injection sign.
4 - capture	Preserve the homed cart origin; add speed/travel/command checks and hysteresis; bound LQR acceleration.
5 - validation	Rebuild, verify pulses and signs with bounded tests, validate balance, then attempt conservative swing-up.

Reading routes

Pages	Purpose
3-9	Changes since September, file map, actual startup, peripherals, encoder and motor behavior.
10-13	Homing, proposed end-stop pins, fault handling and a concrete sign-calibration procedure.
14-18	LQR provenance, the rigid-pendulum model, adjustable pivot, swing-up and handover.
19-22	Prioritized fixes, measured commissioning steps, diagnostics and verification evidence.
23-24	Source references, rebuild instructions and the parameter record for future work.

Throughout this guide, **current** means visible in the inspected source; **proposed** means not yet implemented; **unverified** means a physical measurement or wiring decision is still required. Pin suggestions and numerical geometry examples are not recorded hardware facts.

2. What changed since September

September guide statement	Current source and revised assessment
Motor test runs from the foreground loop.	The foreground loop is empty. The active TIM3 callback updates sensing and control at a configured 1 kHz.
No balancing controller in the project.	lqr.c now computes a four-state acceleration command; swingup.c and swingup.h implement energy-based swing-up.
Direction reversal does not write DIR.	motor.c now stops PWM, writes DIR and inserts a setup delay before restarting. Waveforms still need measurement.
Wrong DMA stream used by encoder code.	encoder.c now reads DMA1_Stream6, matching the configured stream. The APB2 reachability problem remains.
Encoder buffer is not volatile; no wrap handling.	The buffer is volatile; the code chooses the latest DMA slot and wraps angular increments with atan2(sin,cos).
cartpole.m is an uncontrolled nonlinear model.	It is now a linear force-input model, an LQR calculation, closed-loop initial-response plots and a force-to-acceleration gain conversion.
No homing implementation.	Blocking homing now exists and is mandatory at startup, but the limit reader and its call are commented out.

What is still missing

There is no calibrated angular zero, no active end-stop input configuration, no runtime rail supervisor, no explicit general-purpose motor-stop API, and no complete application state machine with arming and latched faults. Cart position is inferred from commanded speed, not measured.

The physical components must be identified again. The folder now includes an 8HY-series motor sheet, a PIB38-series encoder sheet and a DRV8824 sheet, while the motor code names DRV8825. None of these generic documents proves the exact fitted variant or its wiring.

Source anchors: main.c:119-149, 483-513; encoder.c:33-56; motor.c:179-201, 209-291; cartpole.m:1-74. September PDF retained only as historical background.

3. Project map and C essentials

File or directory	Role in this version
test/Core/Src/main.c	Initialization, clock tree, generated GPIO/timer setup, and the TIM3 control callback.
encoder.c / encoder.h	DMA sample selection, count-to-angle conversion and one-sample angular velocity.
motor.c / motor.h	Driver pin control, acceleration integration, STEP pulse generation and blocking homing.
lqr.c / lqr.h	State vector [cart position, cart velocity, upright angle error, angular velocity]; acceleration feedback.
swingup.c / swingup.h	Energy estimate, pump direction, acceleration saturation and an angle-only capture test.
stm32f4xx_hal_msp.c / _it.c	Peripheral clocks, alternate functions, DMA configuration and interrupt routing.
test.ioc	CubeMX configuration to keep aligned with generated initialization code.
Drivers / Core/Startup / *.ld	HAL/CMSIS, reset and interrupt vectors, Flash/RAM linker layouts.
pendulum/cartpole.m	MATLAB design script. It does not execute on the microcontroller.
Debug / .metadata	Copied build products and IDE workspace state; regenerate the build locally.

How the pieces connect

A header declares types and functions; a .c file implements them. A struct groups related state: `motor_drv_t` links the motor and driver handles and holds the commanded speed and estimated position. A pointer such as `motor_drv->motor` accesses the linked object rather than copying it.

TIM3 invokes an interrupt handler; the HAL then calls `HAL_TIM_PeriodElapsedCallback`. This callback is the periodic control task. Keep it bounded and avoid blocking `printf`, delays or homing inside it. The main while-loop currently does nothing.

`volatile` tells the compiler that a value can change outside ordinary code flow. It does not calibrate a sensor, make a multi-field snapshot atomic, fix an invalid DMA route, or guarantee timely samples. A build warning here comes from passing `volatile acc` to a function declared with `float *`.

Source anchors: Core/Inc types; `stm32f4xx_it.c`:218-227; `main.c`:145-149, 483-513. Paths in this guide are relative to `pendulum/test` unless noted otherwise.

4. The actual startup and control path

```
RESET -> HAL_Init -> clock configuration
      -> GPIO + DMA + TIM1/2/3/4 initialization
      -> encoder buffer + TIM4/DMA + TIM1 start
      -> driver wake + motor initialization
      -> motor_homing()
          no hit -> Error_Handler() forever
          success -> start TIM3 interrupt
      -> 1 kHz encoder update -> SWINGUP or LQR
```

The current blocking point

main.c calls motor_homing before starting TIM3. Inside homing_move, s is always set to zero and limit_read is commented out. hit1 therefore remains zero, motor_homing returns zero, and main enters Error_Handler. On functioning timer hardware the first leg runs to the 20,000-period limit; it never observes either end stop.

With HOMING_F=2000 step/s, that is nominally about 10 seconds of attempted motion. At the configured metres/step it represents 6.28 m of requested travel, not a safe rail-distance limit. Actual displacement can be much less if the cart hits an end or the motor loses steps. No wall-clock timeout exists if the timer stops producing update flags.

The callback after homing is repaired

```
update_Encoder(&encoder);
in_range = (165 <= angle && angle <= 195);
enter range: position = 0; control_on = 1;
leave range: control_on = 0;
if control_on: LQR -> motor_acc(acc);
else: swingup_update(...) -> motor_acc(...);
```

This preserves cart commanded velocity on capture, which is an improvement. However, it resets the cart position reference, uses no angular-speed gate, and immediately reverts to swing-up after falling out of the range. The global swingup_t su is zero-initialized; swingup_init(&su) is never called.

Proposed startup

Initialize controller state, establish encoder reference, home with functioning limits, allow the pendulum to settle, seed the estimator from its actual count, and require an explicit armed state before automatic control.

Source anchors: main.c:124-139, 483-513; motor.c:249-275; swingup.c:35-40. This path is source analysis, not a plant run.

5. Pins, timers and units

Signal	Current assignment	Check
STEP	PA0 / TIM2_CH1	Pulse output; no STEP macro in main.h.
DIR	PB12	Low = software CW/positive speed; physical direction unknown.
RESET + SLEEP	PB13	Combined control in code; verify module wiring.
M0 / M1 / M2	PC6 / PB15 / PB14	drv_init selects full step (factor 1).
Encoder channels	PA8 / TIM1_CH1; PA9 / TIM1_CH2	User labels CH2/CH1 are reversed relative to timer channel names.
Left / right end stop	Not assigned	Proposed PC2 / PC3; see page 11.
Debug	PA13/PA14 SWD; PB3 SWO	Keep available for debug.
USART2 / button	PA2/PA3; PC13	UART pins configured, no UART init; button is not an implemented stop.

Clock or timer	Derived configured value
System clock	HSI 16 MHz / 16 x 336 / PLLR 2 = 168 MHz. SYSCLK uses PLLR, not PLLP.
APB1 / APB2	42 MHz each; timer clocks 84 MHz with the configured bus divisors and default TIMPRE.
TIM1 encoder	Prescaler 0, ARR 2399, x4 quadrature: code assumes 600 pulses/rev x 4 = 2400 counts/rev.
TIM2 STEP	32-bit timer, prescaler 0; frequency uses fclk=84 MHz.
TIM3 controller	84 MHz / 84 / 1000 = 1000 Hz; Ts = 1 ms.
TIM4 DMA request	84 MHz / 840 = 100 kHz; this does not make control 100 kHz.

Units: encoder.angle is degrees; rad_angle is radians; w is rad/s. motor_drv.position is estimated STEP pulses; fcurrent is signed pulses/s; motor_acc expects pulses/s². m_per_step=0.0003141593 m/pulse is a calibration assumption, not a measured result.

Source anchors: main.h:62-89; main.c:38-43, 170-192, 218-233, 269-297, 327-330, 372-375; test.ioc; MSP:102-110, 198-205.

6. Encoder: routing and calibration

A confirmed DMA bus mismatch

HAL_DMA_Start uses &TIM1->CNT as the peripheral source while hdma_tim4_up is configured as DMA1_Stream6/channel 2, peripheral-to-memory. TIM1 is on APB2. DMA1's peripheral port reaches APB1 only, so the configured route cannot provide the intended TIM1 samples. A valid TIM4 DMA request does not change source-bus reachability. [R1]

The code disables the DMA IRQ and does not check transfer-error flags or the return value of HAL_DMA_Start. update_Encoder then treats the RAM buffer as valid. Frozen or invalid readings must inhibit control, not be accepted as a stationary pendulum.

Recommended first correction

Read TIM1->CNT once in the TIM3 control tick and pass that count into an encoder-update function. The CPU can read it directly; 1 kHz counter reads do not require a 100 kHz DMA stream. Remove the unused DMA sampling path consistently from startup and CubeMX. If DMA is a course requirement, redesign the request/stream/source route using the F446 reference manual and verify it independently.

Calibrate before interpreting zero

An incremental encoder does not know “down” at power-up. The current count begins at zero wherever the rod happens to be. reset_Encoder only resets struct fields: it does not reset TIM1->CNT or establish a reference offset. Set the downward reference while stationary, record its count, then apply a sign and modular offset to every sample.

```
N = measured counts per mechanical revolution
phi = wrap_0_2pi(sign_enc * 2*pi*(count-down_count)/N)
theta = wrap_minuspi_pi(phi - pi)
omega = wrapped_delta(phi, previous_phi) / Ts
```

Set previous_phi to the first valid calibrated sample and initially set omega to zero. In the present code, a first reading at 180° is compared with the initialized 0° and produces |omega| about 3141.6 rad/s. Homing can move the rod before the first callback, so this is a realistic startup condition once homing is repaired.

Source anchors: main.c:121-129; MSP:164-179; encoder.c:11-30, 33-56. Hardware bus claim: ST AN4031 section 2.2.1, page 21 [R1].

7. Encoder velocity and data quality

What is correct in the new code

The buffer is volatile and update_Encoder reads DMA1_Stream6->NDTR rather than an unrelated stream. It retries if NDTR changes while reading the two slots. It computes the difference between successive control samples and wraps the difference through atan2(sin,cos). The focused host test confirms the 2399-to-0 transition behaves as one count, rather than a full revolution.

Why the raw derivative is too coarse

```
one count = 2*pi/2400 = 0.00261799 rad = 0.15 deg
one count / 1 ms = 2.61799 rad/s
LQR omega gain = 69.6459 m/s² per rad/s
velocity term from one count/ms = 182.33 m/s²
```

The code has no angular-velocity filter. A slowly moving rod therefore gives a sequence of zeros and large velocity pulses. With the present LQR gain, one count per tick produces about 580,383 step/s² from the velocity term alone. The swing-up limit is only 2,000 step/s²; LQR has no corresponding acceleration bound.

Use an estimator with a measured noise-delay tradeoff: for example a fixed time window, a low-pass filtered count derivative, or an observer fitted to the model. Keep unwrapped count increments for rate estimation while using wrapped angles for orientation. Validate with real logs at rest, slow rotation and free oscillation.

Checks for both polling and DMA designs

Check	Acceptance evidence
Scale	One mechanical turn gives the expected count change; current N=2400 is verified, not inferred from a filename.
Electrical interface	Exact encoder model, supply and output type identified; logic levels and pull-ups compatible with MCU inputs.
Reference and sign	Down is 0°, upright is 180°; the chosen positive lean is repeatable.
Freshness and faults	A valid transfer/sample mechanism is checked; frozen data is not confused with confirmed stationary motion.
Timing	Actual control interval and worst-case execution time measured; estimator delay included in capture tests.

The two-slot NDTR check is an improvement, not a universal coherence guarantee: DMA can lap the reader, a transfer can fail, and startup may read an unfilled slot. Direct counter reads simplify this project considerably.

Source anchors: encoder.h:12-24; encoder.c:38-56; lqr.c:14-20. Evidence: output/review/host_checks.txt and numerical_checks.json.

8. Motor commands and cart position

What motor_acc actually does

```
dt = 1 / fs
v = clamp(fcurrent + acc*dt, -fmax, +fmax)
fcurrent = v
position += v*dt
F = abs(v)
if F < 5 Hz: stop STEP PWM
else: set DIR and program ARR/CCR for STEP
```

The current driver sets full-step mode and assumes 200 motor steps/revolution. Each call integrates acceleration once. A zero acceleration holds the existing speed: **motor_acc(0) is not a stop**. Negative acceleration is braking only while the current velocity is positive; after crossing zero it accelerates in the opposite direction.

Improvements that should remain

ARR and CCR preload are enabled, the STEP high pulse has a fixed nominal 2.5 μ s duration, and a direction change writes the pin while PWM is stopped before restarting. At 84 MHz the configured high pulse is 211 ticks, about 2.512 μ s. These changes address old direction and timer-period problems. Validate minimum high/low pulses and DIR setup/hold during stop, reversal and software update events. [R3]

Position is a command estimate

position is the integral of requested speed. It is not a count of actual rising STEP edges and it cannot detect missed motor steps, belt slip or an obstruction. It advances even when PWM is off below 5 Hz: the host check gives 0.004 estimated step per tick at 4 step/s with no emitted PWM. Timer preload/restart behavior can cause additional differences between requested and emitted pulses.

For usable software rail limits, preserve an absolute position established by homing and preferably count emitted pulses with direction. State explicitly that even pulse counting does not prove mechanical motion; a separate cart sensor would detect tracking failures. Use a separate x_ref for any desired balance point instead of zeroing the position estimate on capture.

Configured conversion	Result; not a demonstrated actuator capability
m_per_step	0.314159 mm per pulse; equivalent to a 10 mm pulley radius at 200 full steps/rev with direct drive.
fmax = 8000 step/s	2.513 m/s requested maximum speed.
Swing acceleration = 2000 step/s ²	0.6283 m/s ² . LQR is not limited to this value.
Microstep changes	Recompute metres/pulse and pulse speed/acceleration limits; changing M0/M1/M2 alone breaks scaling.

Source anchors: motor.c:71-102, 105-202; motor.h:19-20; lqr.h:4. DRV8825 timing is conditional on that chip being fitted [R3].

9. Homing needs more than two macros

Existing sequence

motor_homing first moves CCW until either switch, then CW until the other switch while counting timer update events, then CCW for half the measured count. It sets position and fcurrent to zero at the nominal center and returns the measured span. main checks only whether that span is nonzero and discards it.

Restoring limit_read and adding pin definitions is necessary, but does not complete the behavior. The routine starts PWM before checking the switch state. If the cart starts on the end toward which CW would push, the first phase can immediately accept that switch and the second phase can move farther into it. Both switches active is not rejected as an impossible state. The return-to-center phase passes watch=0 and ignores both inputs.

Proposed homing state machine

State	Required behavior
PRECHECK	Read left/right independently before motion. Both active -> fault. Verify configured polarity and a valid GPIO setup.
RELEASE	If a switch is already active, move away from that identified physical end under a small bounded release budget; require it to clear.
SEEK LEFT	Move in the calibrated physical left direction, with ramped speed, maximum time and travel. Unexpected right activation -> fault.
BACKOFF + LATCH	Back off, require release, then approach slowly for a repeatable left reference. Debounce or qualify the latched event.
MEASURE RIGHT	Traverse with the same protections, record the right limit and confirm a plausible span.
CENTER	Move to a chosen interior reference with both switches supervised; retain usable span and reserved margins.
READY or FAULT	Stop pulses explicitly, reset commanded motion state, publish homed validity, and require deliberate recovery after a fault.

Avoid an instantaneous 2000 step/s startup or reversal until that speed is validated for the loaded motor. Count actual pulse events reliably; UIF is a flag, not a queue, so multiple elapsed periods can collapse into one observation. Also account for the first edge when PWM starts. A wall-clock timeout must remain effective if pulses stop.

Keep the rail reference

Store the measured span and absolute home origin for runtime bounds. A balance reference is a separate variable. Do not erase the homed coordinates when the rod reaches upright.

Source anchors: motor.c:209-291; main.c:135-139, 492-497. All states above are proposed, not present in the current firmware.

10. Proposed end-stop pin assignment

The current source leaves both end stops unassigned. PC2 and PC3 are unused in test.ioc and avoid STEP, DIR, mode pins, the encoder, UART and SWD. They are a practical proposal for the NUCLEO-F446RE; confirm access on the actual board and that colleagues have not wired them elsewhere.

Function	Proposed GPIO	Nucleo connector [R2]
LIMIT_LEFT / LIMIT_A	PC2; optional EXTI2	ST morpho CN7, pin 35
LIMIT_RIGHT / LIMIT_B	PC3; optional EXTI3	ST morpho CN7, pin 37

Use physical names LEFT and RIGHT, defined while standing at the plant in a recorded viewing direction. Do not infer left from CW: motor coil order, pulley mounting and belt arrangement determine the physical direction.

Choose polarity from the actual contacts

Dry-contact wiring to GND, pull-up to 3.3 V	Released	Pressed / actuated
Normally open contact (NO)	HIGH	LOW: active-low
Normally closed contact (NC)	LOW	HIGH: active-high; an open wire also reads active

These are wiring examples for passive contacts, not assumptions about the fitted devices. A powered sensor or switch module may have a different voltage/output circuit and needs the matching interface. Do not connect an unverified powered output directly to the MCU. The current commented code assumes active-low, which would be wrong for the NC example above.

Implementation changes that must stay together

1. Configure PC2 and PC3 in CubeMX, label them and select appropriate input bias. Regenerate main.h and GPIO initialization.
2. Restore the reader using the chosen polarity; reject contradictory inputs.
3. Feed both inputs into homing and into every runtime motion mode.
4. If using EXTI, implement and enable EXTI2/EXTI3 handlers and their stop/fault route. Still read the levels at boot: a switch already active does not require a new edge.
5. With motion inhibited, verify each input while pressing and releasing the actual switches, and test a disconnected cable for the chosen wiring.

Status

PC2/PC3 are proposed assignments only. This review has not changed test.ioc, main.h or the firmware, and the switch contact type remains unverified.

Source anchors: No LIMIT_* definitions exist in current main.h; motor.c:211-222 contains the disabled active-low reader. Connector positions: ST UM1724 Rev 17, Table 29, page 59 [R2].

11. Stop behavior and rail supervision

A fault must stop the pulse source

Error_Handler disables interrupts and loops. TIM2 is a hardware pulse generator and can keep running without CPU interrupts. The fault handlers also loop without explicitly stopping STEP. Homing normally stops PWM before returning its failure, but this does not make the general fault path safe during motion.

Provide a small, bounded stop routine that disables the STEP output/timer, forces a known inactive pulse state as appropriate for the pin configuration, sets fcurrent to zero, clears pwm_on, and marks the motion state stopped. Record the fault reason. Decide separately whether the driver should hold torque or enter sleep; disabling torque does not mechanically brake a moving cart.

Supervise every mode

Homing, swing-up, balance and manual tests need the same checks for end-stop activation, invalid sensor data, invalid calibration, implausible state, timing failure and motion budget expiry. Outside the controlled release phase of homing, an activated end stop should latch a fault. Do not automatically rearm after a switch clears.

```

fault or disarmed -> explicit stop, no controller output
homing -> bounded homing action + switch checks
swing-up -> energy action + common motion bounds
balance -> LQR action + common motion bounds
all modes -> one validated actuator update per tick

```

Software limits need a credible position

Store the rail span from homing and reserve a margin before each physical end. Use measured braking capability, sensor/processing delay and position uncertainty. For an ideal constant-deceleration estimate, stopping distance is $v^2/(2*a_{brake})$; add $|v|*latency$ and a measured uncertainty margin. This is a planning model, not a guarantee for an open-loop stepper that may stall.

SWINGUP_KX and SWINGUP_KV are both zero, so current swing-up has no centering term at all. Even nonzero centering gains cannot replace hard limits, a braking envelope or a failed-capture policy. Returning to high-energy swing-up immediately after every loss of balance can use up remaining rail travel.

Hardware checks matter

Test both end stops and the independent means of removing motor drive before energetic experiments. Also verify what STEP does when the debugger halts the CPU; no explicit TIM2 debug-freeze configuration was found in application code.

Source anchors: main.c:522-529; stm32f4xx_it.c fault handlers; swingup.h:55-59; main.c:499-512; motor.c:146-152.

12. Establish the signs systematically

One convention for the whole project

Choose x positive to the right as viewed from a marked side of the plant. Choose upright angle θ positive when the pendulum center of mass leans right. Use $\theta=0$ upright and $\phi=\text{wrap}(\theta+\pi)$, so $\phi=0$ down and $\phi=\pi$ upright. Angular velocity is the derivative of that same signed angle.

```
X_COM = x + d*sin(theta) = x - d*sin(phi)
Y_COM = d*cos(theta)      = -d*cos(phi)
J*theta_dd = m*g*d*sin(theta) - m*d*a*cos(theta)
J*phi_dd    = -m*g*d*sin(phi) + m*d*a*cos(phi)
```

Bounded calibration sequence

1. **Motor sign:** with feedback disabled and ample travel, command a small, limited pulse motion. Record which DIR level moves right. Define MOTOR_SIGN so a positive physical acceleration produces rightward acceleration. Apply the same sign to cart position and velocity conversion; do not flip only the actuator output.
2. **Encoder sign:** with the motor inhibited, move the rod gently through upright toward the chosen right. After calibration, θ must increase and ω must be positive during that motion. Define ENCODER_SIGN and the reference count.
3. **LQR sanity check:** at $x=v=\omega=0$ and a small positive θ , the present C gains command positive acceleration. This should move the cart beneath a rightward lean.
4. **Dynamic sign check:** apply a small, bounded known cart acceleration and compare the initial angular response with the equation above. Then verify swing-up energy change in a low-amplitude trial.

The swing-up sign follows the chosen convention

```
E = 0.5*J*omega^2 + m*g*d*(1-cos(phi))
E_dot = m*d*a*omega*cos(phi) [ideal, no friction]
For E < E_ref: a must have sign(omega*cos(phi)).
```

The current implementation has a leading minus: $a_{\text{step}} = -\text{SWINGUP_SEGNO} * K * (E_{\text{ref}} - E) * \text{sign}(\omega * \cos(\phi))$. If the encoder and positive motor command follow the convention above, SWINGUP_SEGNO must be -1 to inject energy. Under a different coordinate mapping the value changes. Therefore +1 in the current header is not a validated physical sign, and changing it alone cannot repair inconsistent LQR or cart-state signs.

Perform these checks with measured motion, not by increasing gains until something seems to work. Keep physical coordinate signs separate from driver CW/CCW names and switch electrical polarity.

Source anchors: lqr.c:14-20; motor.c:129-135; swingup.c:69-78; swingup.h:42-45. Equations here are independently derived for the declared coordinate convention.

13. A model that matches the actuator

Use COM distance and pivot inertia separately

Let m be pendulum mass, d the pivot-to-center-of-mass distance and J its moment of inertia about the pivot. Define the equivalent dynamic length $L_{\text{eff}} = J/(m*d)$. L_{eff} governs the angular response to gravity and imposed cart acceleration; d governs potential energy. They are equal only for a point mass at distance d on a massless rod.

For an ideal cart-acceleration input a , neglecting friction, the nonlinear upright equation is $\ddot{\theta} = (g/L_{\text{eff}})\sin(\theta) - (a/L_{\text{eff}})\cos(\theta)$. The cart equation is $\ddot{x} = a$. This is appropriate only while the stepper tracks the commanded trajectory without losing steps.

```
State z = [x, v, theta, omega]^T
z_dot = A_acc*z + B_acc*a
A_acc = [0 1      0 0]
        [0 0      0 0]
        [0 0      0 1]
        [0 0 g/L_eff 0]
B_acc = [0, 1, 0, -1/L_eff]^T
```

If you retain a force-input rigid-rod model

For a frictionless cart mass M and upright small-angle coordinates, let $D = (M+m)*J - (m*d)^2$. The general force model is:

```
A_force = [0 1      0 0]
           [0 0      -m^2*g*d^2/D 0]
           [0 0      0 1]
           [0 0 (M+m)*m*g*d/D 0]
B_force = [0, J/D, 0, -m*d/D]^T
```

The present MATLAB A/B matrices have the familiar point-mass form but substitute L_{eff} for length. That substitution reproduces the angular acceleration model after the script's ideal force-to-acceleration conversion. It does not in general reproduce the actual force-driven dynamics of a distributed rod with physical masses M and m .

Practical design path

Measure $J/(m*d)$, metres per pulse, actuator tracking and delay. Design and test the acceleration-input controller directly at the actual sample time. Keep a nonlinear, limited-travel simulation for swing-up and capture; a linear initial-response plot cannot validate a full swing.

Source anchors: cartpole.m:4-16, 66; lqr.c:16-20. General energy-shaping background: MIT Underactuated Robotics [R4]; the equations and state order above are derived for this guide.

14. LQR: the gains are traceable

The current MATLAB workflow

cartpole.m uses $l1=0.35$, $l2=0.65$, $m=0.1$, $M=1.0$ and $L=0.7055556$. It forms a linear force-input model, uses $Q=\text{diag}(100,1000,2000,1000)$ and $R=1$, and calls `lqr`. It then converts the gain to an acceleration gain using $K_a=K/M+[0,0,m*g/M,0]$. LQR convention is $u=-K*z$. [R5]

Gain	Independent result
Force gain K	[-10.0000, -38.2160, -236.6478, -69.6459]
Converted acceleration gain K_a	[-10.0000, -38.2160, -235.6668, -69.6459]
Gain stored in <code>lqr.c</code>	[-10.0000, -38.2160, -235.6668, -69.6459]

The C gain matches K_a to its printed precision. The code's minus sign and division by `m_per_step` are consistent with the script's converted acceleration input. The MATLAB comment " $a = K_a * dx$ " omits the minus sign and should be corrected to $a = -K_a * dx$.

A mathematical consistency check, not a plant validation

Ideal $A_{\text{acc}} - B_{\text{acc}} * K_a$ poles:
 $-55.03445, -0.316565, -2.571853 \pm 1.168873j$

These stable poles were independently calculated with a NumPy Hamiltonian solution of the continuous Riccati equation; the maximum Riccati residual was about $5.7e-10$. This confirms internal model/gain consistency. It does not validate physical signs, actual geometry, noisy derivatives, a sampled implementation, saturation, rail travel or actuator tracking.

What needs changing around LQR

The C function has no acceleration saturation. At a stationary 15° lean, its angle term alone asks for about 61.7 m/s^2 . The 8000 step/s speed clamp is not an acceleration limit and cannot stop a single large command from demanding an abrupt velocity change. Apply measured acceleration/braking limits to both controllers, then re-evaluate the capture region.

The MATLAB tail labelled "OSSERVATORE DI STATO" is not an observer. It computes $E_{\text{rif}} = m * g * L$ and then overwrites K with 1200. If L is the dynamic length, that E_{rif} is also not the physical down-to-up energy. Use descriptive names for force gains, acceleration gains and swing energy gain; export calibrated parameters consistently.

Source anchors: `cartpole.m:47-74`; `lqr.c:12-20`; `main.c:504-511`. Independent calculations and build evidence are under `output/review/`.

15. What adjusting the pivot changes

If the pivot moves along the rod

Moving the pivot closer to the rod's center of mass reduces the gravitational energy barrier, so it can help. It also changes inertia, angular response and how effectively cart acceleration drives the rod. It is not a universal "closer is always easier" adjustment. Recalculate the model and retune for each pivot setting.

For a uniform rod with lengths l_1 and l_2 on opposite sides of the pivot, with total rotating mass m , the following are independently derived. They exclude clamps, hubs, counterweights and encoder shaft inertia unless those are added explicitly.

```
rod length = l1 + l2
d = abs(l2-l1)/2
J = m*(l1^2 - l1*l2 + l2^2)/3
L_eff = J/(m*d)
E_up - E_down = 2*m*g*d
```

Quantity	Example from current MATLAB numbers
$l_1 / l_2 / m$	0.35 m / 0.65 m / 0.10 kg; unverified plant values
Pivot-to-COM distance d	0.1500 m
Pivot inertia J	0.0105833 kg·m ²
Equivalent dynamic length L_{eff}	0.7055556 m
Physical down-to-up energy	0.2943 J
Current C energy reference	1.38439 J, because SWINGUP_L=0.7056 is used as COM distance

The formula for L in cartpole.m is exactly L_{eff} for this rod interpretation. Copying it into SWINGUP_L, whose comment calls it a pivot-to-COM distance, confuses two different lengths. Under the example geometry it overstates the potential-energy barrier by about 4.70 times; SWINGUP_J=0 also removes kinetic energy entirely.

As d approaches zero, the energy barrier approaches zero but cart-to-rotation coupling $m*d/J$ also approaches zero. At the exact center of mass the ideal balanced rod cannot be swung by horizontal cart acceleration through this mechanism, and it no longer has a unique gravity-defined down direction. Avoid treating the singular L_{eff} formula as a useful endpoint.

If the whole assembly is raised or lowered

If the pivot stays at the same point on the rod, d and J do not change. A fixed vertical translation adds the same constant to both potential energies, so their difference remains $2*m*g*d$. Clearance may improve enough to permit a full swing, but the ideal energy requirement does not decrease. The type of adjustment and actual geometry remain to be confirmed.

Source anchors: cartpole.m:2-5; swingup.h:23-36. Numerical example assumes a uniform rod and is not a measurement of the photographed plant.

16. Swing-up: implementation and fixes

What the current controller computes

```
E = 0.5*SWINGUP_J*w*w + m*g*L*(1-cos(phi))
E_ref = 2*m*g*L
e = wrap(phi-pi)
if abs(e) < 15 deg: return CAPTURE (no motor call)
update stored sign from w*cos(phi), outside +/-0.05
a = -SEGN0*1200*(E_ref-E)*stored_sign
a -= KX*position + KV*fcurrent
a = clamp(a, -2000, +2000)
motor_acc(a)
```

K is expressed in step/s^2 per joule, so a mechanical parameter or microstep change changes the effective gain. $KX=KV=0$ disables cart centering and damping. The controller reports energy, angle error and last acceleration in su, but no logging pipeline records them.

Missing initialization prevents a deliberate start

swingup_init sets segno=+1 to provide an initial kick at zero angular speed. It is declared and defined but never called. The global su starts with segno=0, so a motionless hanging pendulum produces exactly zero acceleration. An incidental oscillation after homing might eventually set the sign, but that is not a defined startup method. Call the initializer before control starts and reset it on the intended new-attempt transition.

The energy observer is currently incomplete

$SWINGUP_J=0$ means the reported energy does not change when angular speed changes at a fixed angle. A fast upright crossing looks energetically the same as a stationary one. Supply the real positive pivot inertia and the COM distance; do not reuse L_{eff} as d. Check the energy estimate with a gentle free swing: it should approximately decay with friction when the cart is fixed.

Bound the experiment as well as the command

After correcting the sign and model, add a finite startup-kick duration, a maximum attempt time, rail bounds and a recoverable failed-capture policy. With $KX=KV=0$, the initial stored-sign action can continue driving the cart if the rod does not respond. A per-sample acceleration clamp alone does not bound velocity, total travel or experiment duration.

No swing-up success is established

The host checks establish arithmetic and control-path defects. They do not simulate the nonlinear pendulum, motor torque, friction, rail limits or full swing-up. Validate the mechanism and balance capture region before enabling energetic attempts.

Source anchors: swingup.h:21-59; swingup.c:35-88; main.c:64, 124-139, 509-512. Controller background: [R4].

17. Capture and preserve the rail origin

Current handover has only an angle condition

main.c selects LQR whenever the encoder angle is between 165° and 195°. swingup_update independently uses a strict 15° wrapped window, and its return value is ignored by main. Keep one authoritative transition decision; independent thresholds can diverge when edited and can leave a tick without the expected actuator update.

The current decision accepts any angular speed and any cart state. The host test confirms an upright state at 100 rad/s returns capture. That is a diagnostic input, not a measured plant speed, and demonstrates the missing gate. Entering the angle window does not mean the rod is slow enough to catch.

Proposed capture conditions

Condition	Reason
Small wrapped upright error	The LQR model is local to upright.
Recoverable angular speed	A fast crossing can leave the capture region before the motor responds.
Cart inside an interior capture zone	Allow enough travel for the upcoming corrective motion.
Recoverable cart speed	Check a braking margin in the current direction.
Valid sensor and home reference	Do not switch based on uncalibrated or stale states.
Feasible bounded LQR command	A nominal controller command may exceed the actuator's usable envelope.
Persistence and hysteresis	Require entry conditions to persist; use a separate wider exit condition to avoid chatter.

The numerical thresholds must come from bounded balancing trials and nonlinear simulation with the actual actuator limits. “Enter at 10°, leave at 15°” can illustrate hysteresis but is not a validated setting for this plant.

Keep physical state continuous

The present entry code sets motor_drv.position=0 each time control turns on. It preserves fcurrent, which is good, but discards the homed rail center. Repeated captures can move the virtual origin toward a rail end. Keep absolute position and velocity unchanged. If balancing at the capture location is desired, set a separate x_ref and constrain it to a safe interior region.

On loss of balance, choose between a bounded recovery attempt and a latched stop according to the remaining travel and velocity. The present immediate return to swing-up is not supervised. Common acceleration/speed/travel constraints must apply on both sides of the handover.

Source anchors: main.c:490-513; swingup.c:61-64. The host harness executes an extracted copy of the actual callback to verify the origin reset.

18. Fix list with source anchors

ID / priority	Confirmed issue and recommended correction
F1 / blocking	motor.c:249-254; main.c:135-139. Limit read is replaced by s=0, so mandatory homing always fails after attempted motion. Configure inputs, restore reads and supervise every phase.
F2 / blocking	main.c:125; MSP:164-179. DMA1 peripheral source is TIM1/APB2. Read TIM1 in the control tick or redesign a supported DMA route; check errors and sample validity.
F3 / high	main.c:124-139; swingup.c:35-40. Missing swingup_init leaves the start direction zero. Initialize it before enabling control and define reset/rearm behavior.
F4 / high	swingup.h:23-36. J=0; L is a placeholder consistent with MATLAB dynamic length rather than COM distance. Measure/update J and d; derive the reference energy.
F5 / high	main.c:490-511; motor.c:134-135. Capture uses only angle and resets the homed position; swing-up has no active rail supervisor. Keep absolute state and introduce validated capture/travel rules.
F6 / high	lqr.c:14-20; encoder.c:49-56. Raw 1 ms velocity plus no LQR acceleration bound produces very large commands. Seed/filter velocity; apply common measured actuator limits.
F7 / high	main.c:522-529; fault handlers. Fault paths do not explicitly stop TIM2 output. Implement a latched stop path and verify behavior on CPU halt/fault.
F8 / high	motor.c:238-283. Homing lacks initial-switch discrimination, release/debounce, ramping, wall-clock timeout and checks on centering. Rework as bounded directional states.

Additional corrections and measurements

F9: encoder.c:11-19 only resets software; establish physical offset and sign, and align ARR with measured counts/rev. **F10:** motor.c:134-152 integrates position without actual pulse accounting, including stopped deadband. **F11:** signs and metres/pulse remain physical calibrations. **F12:** main.c:506 discards volatile through the LQR pointer; compute into a local float, then copy to the volatile debug variable. **F13:** debug.h/lqr.h lack include guards; add them. **F14:** MATLAB line 67 needs the minus sign, and lines 71-74 need correct energy definitions and distinct variable names.

The supplied Debug makefile contains an absolute /home/vito path. Regenerate build files in CubeIDE; do not repair generated object dependencies by hand. Saved trace settings use 84 MHz HCLK although SYSCLK is 168 MHz; correct this if enabling SWV. None of the findings above was silently patched into the firmware by this review.

19. A staged route to a working plant

Stage	Work	Pass condition
1. Offline corrections	Resolve F1-F8; keep CubeMX and source synchronized; make signs/calibration explicit.	Fresh ARM build and diagnostic tests pass without unresolved warnings.
2. Inputs only	Keep motion inhibited. Test end stops, encoder counts, reference, direction and electrical levels.	Each physical input is identified; impossible/disconnected states handled as designed.
3. Actuator only	Use limited pulse moves, low verified speed/acceleration and an independent stop. Measure STEP/DIR and actual displacement.	Correct direction and scaling; bounded stop/reversal behavior; no missed steps within chosen envelope.
4. Homing	Test starts in the middle, on either switch, and fault cases; log measured travel.	Repeatable center/span; release works; unexpected switch or timeout always stops.
5. Estimator and model	Free-swing tests; measure pivot geometry and oscillation period; inspect angle and velocity logs.	Model ratio and sign agree; derivative noise/delay acceptable; origin maintained.
6. Manual upright balance	Place rod near upright; enable a restricted capture region and common actuator limits.	Repeatable small-disturbance recovery with cart well inside the travel envelope.
7. Partial swings	Validate energy sign and centering with bounded attempts.	Energy rises when requested; travel remains bounded; aborts and limits work.
8. Full swing-up/capture	Use the validated model, limits, handover and chosen pivot setting.	Repeated successful capture and acceptable recovery/stop on failed attempts.

Record evidence at every stage

Log timestamp, mode, raw count, calibrated phi/theta, raw and filtered omega, cart estimate, x_{ref} , commanded speed/acceleration, applied limits, left/right switches and fault reason. For swing-up include E , E_{ref} and stored pump sign. Decimate or buffer telemetry so logging does not delay the 1 kHz callback.

No numerical setting in this guide should be labelled “validated” until it has a measurement or test log. Start with one known mechanical configuration; after changing the pivot, update the parameter record and repeat the relevant model and capture tests.

Source anchors: Proposed workflow based on the source findings. No stage beyond offline build and diagnostic checks was performed in this review.

20. Troubleshooting this version

Observed symptom	First explanation to check
Cart moves at startup, then firmware stops	Current homing never reads end stops; it reaches its period budget and enters Error_Handler.
Motor does not start swing-up after homing repair	su.segno remains zero without swingup_init; also check invalid/frozen encoder data and the armed state.
Angle remains constant while rod turns	First inspect TIM1->CNT directly. If it changes but RAM does not, inspect the invalid DMA route and error flags.
Angle scale is wrong	Confirm exact encoder model and counts/rev. The code assumes 600 pulses/rev and x4 decoding.
Large command on first control tick	The estimator starts with previous angle zero, even when the sampled rod is near upright after homing.
Violent commands or chatter near upright	Raw one-count/ms velocity is 2.618 rad/s; filter and validate delay, then check acceleration limits and signs.
Swing-up damps instead of pumps	Check consistent cart/encoder signs using the energy derivative; do not compensate by increasing K.
Fast top crossing, failed capture	No angular-speed gate; energy model lacks J; no feasible-command or remaining-travel test.
Cart reference drifts between attempts	position is reset on capture and otherwise integrates commanded speed, including stopped deadband.
Motor continues after software fault or halt	Timer PWM can continue autonomously; verify explicit stop and debugger freeze behavior.
No printf in serial terminal	debug.c sends to ITM/SWO; USART2 has pin setup only, with no UART initialization or transmit backend.
Copied project does not build locally	Import the inner test project and regenerate Debug paths; the copied makefile refers to /home/vito.

Separate software proof from hardware proof

A successful build proves that the translation units compile and link together. A host diagnostic proves a particular arithmetic/control-path behavior under its test inputs. Neither establishes GPIO voltage, pulse integrity, motor torque, true cart position or a stable physical closed loop.

Source anchors: Evidence and commands: `output/review/build_log.txt`, `host_checks.txt`, `verify_snapshot.py` and `run_host_checks.py`.

21. What was verified here

Fresh ARM build: passed with one warning

The installed STM32 toolchain (GNU Tools for STM32 14.3.rel1) compiled 27 source/assembly units from the current tree and linked a fresh Cortex-M4 hard-float ELF. The build used the project Flash linker script and equivalent Debug flags, with output isolated under tmp/review-build. The shipped ELF was not used as build evidence.

```
text: 24,180 bytes   data: 12 bytes   bss: 2,100 bytes
warning: main.c:506 discards volatile qualifier
         when passing &acc to lqr_controller(float *)
```

CubeIDE regeneration itself was not run. The script bypasses the copied makefile's stale absolute path while compiling the same source inventory. No download/flash command was executed.

Focused host diagnostics: passed

Check using original C modules	Observed result
Zero-initialized swingup_t, motionless rod	No kick; after swingup_init, about -1661.265 step/s ² with current placeholders.
Same angle, w=1 then w=10 rad/s	Exactly equal reported energy because J=0.
Upright, w=100 rad/s	Capture returned despite high angular speed.
Encoder count wrap 2399 to 0	Correct one-count forward velocity, about 2.618 rad/s.
First sample at 180°	Artificial w about 3141.592 rad/s.
Motor speed 4 step/s	PWM remains off while estimated position increases.
motor_acc(0) at 100 step/s	Maintains speed rather than stopping.
Actual callback, entering LQR at position 321	Homed position is discarded.
LQR with one-count/ms angular velocity	About 580,383 step/s ² from the velocity term.

The host harness uses small HAL stubs and an extracted copy of the current callback. It does not emulate DMA buses, timer shadow registers, GPIO timing or the physical plant. The DMA finding is checked against ST documentation, not inferred from the host test.

Independent model/gain calculations are in numerical_checks.json; a SHA-256 manifest records the exact reviewed source/configuration snapshot. Source hashes were checked again after document generation.

22. Sources and reproduction

Local source of truth

The current C and header files under Embedded-Course-Dma/pendulum/test/Core, test.ioc, the generated HAL/MSP/interrupt paths, linker scripts and build metadata were reviewed. The active motor/encoder/LQR/swing-up paths were inspected in detail. The unrelated Tartarus exercise and bare-metal LED example do not participate in this firmware.

The old STM32_Inverted_Pendulum_Project_Guide.pdf supplied the historical baseline. Its source walkthrough and “not implemented yet” statements are superseded by this revision. The older paper discussion is not used as authority for the current gain: the current cartpole.m and an independent Riccati calculation establish that provenance.

The folder’s 8HY.PDF, 1-1zp1131z4.pdf and drv8824.pdf identify families of components only. Confirm fitted part markings and wiring. IMG-20260907-WA0006.jpg is reused as an unmodified reference image.

Primary external references checked on 8 October 2026

[R1] ST, AN4031, section 2.2.1, page 21. DMA1/2 peripheral-port reachability. [Open source.](#)

[R2] ST, UM1724 Rev 17, Table 29, page 59. NUCLEO-F446RE ST morpho pins. [Open source.](#)

[R3] TI, DRV8825 datasheet, timing table, page 7. Conditional reference if this is the fitted driver. [Open source.](#)

[R4] MIT, Underactuated Robotics, Acrobots, Cart-Poles and Quadrotors. Energy shaping and local stabilization background. [Open source.](#)

[R5] MathWorks, lqr. Continuous LQR convention and gain definition. [Open source.](#)

Reproduce the offline checks

```
cd "<project folder>"
<bundled-python> output/review/verify_snapshot.py
python3 output/review/run_host_checks.py
<bundled-python> output/pdf/build_guide.py
```

verify_snapshot.py uses the installed ARM compiler path and NumPy. build_guide.py uses ReportLab, Pillow and pypdf from the bundled runtime. The scripts are documentation/audit tools, not firmware files. The guide has also been rendered to page images and visually checked.

23. Parameter and decision record

Fill this record on the lab PC and keep it with the code. Values below are current assumptions or open decisions; they are not confirmed measurements. A changed pivot setting needs a new record.

Item	Current code / proposed value	Record after verification
Board	NUCLEO-F446RE	Board revision: _____
Driver	Code: DRV8825; supplied PDF: DRV8824	Chip / supply / current limit: _____
Stepper	200 full steps/rev; full-step mode	Exact model / mode: _____
Encoder	600 pulses/rev x4 = 2400 counts/rev	Exact model / measured N: _____
Left / right end stop	Proposed PC2 / PC3 (CN7 35 / 37)	Pins / NO-NC / active level: _____
Positive cart direction	Proposed right, from a marked viewing side	DIR level / MOTOR_SIGN: _____
Positive angle	Proposed COM lean right from upright	ENCODER_SIGN / down count: _____
Metres per pulse	0.0003141593 m/pulse	Measured calibration: _____
Rail limits	Not retained by current main	Home span / interior bounds: _____
Pivot setting	MATLAB l1=.35 m, l2=.65 m	Actual lengths / adjustment type: _____
Mass / COM / inertia	m=.1 kg; swing d=.7056 m; J=0	Measured m / d / J: _____
Control / velocity filter	1 kHz; no filter	Measured Ts / estimator: _____
Motion limits	8000 step/s; swing 2000 step/s ²	Validated speed / accel / brake: _____
Capture / exit	165-195° only	Angle / speed / travel / dwell: _____
Build and test evidence	Source hash manifest and offline logs	Commit or snapshot / lab log: _____

Suggested shared context for the next task: "Use this October guide and the current source. Swing-up exists but is not hardware-validated. Fix the numbered findings, confirm end-stop wiring and coordinate signs, and calibrate the chosen pivot geometry before enabling automatic motion."